

10X

Your AI Solutions for Real-World

(Data Science Roadmap to Excellence)

100

Python Libraries

for Impactful Machine Learning
and Deep Learning



by Maryam Miradi

Contents

1. Business Understanding

- **Steps**
- **Python Libraries**

2. Data Understanding & EDA

- **Steps**
- **Python Libraries**

3. Data Preparation

- **Steps**
- **Python Libraries**

4. Feature Engineering

- **Steps**
- **Python Libraries**

5. Modeling

- **Steps**
- **Python Libraries**

6. Model Fine-Tuning

- **Steps**
- **Python Libraries**

7. Evaluation / Comparison

- **Steps**
- **Python Libraries**

8. Deployment

- **Steps**
- **Python Libraries**

9. AI Explainability

- **Steps**
- **Python Libraries**

10. AI Fairness

- **Steps**
- **Python Libraries**

Thank you for downloading this Guide!

If You have any these questions then this Guide is for you.

1. Misaligned Business Objectives

- "How does this model or solution directly address our core business objectives?"
- "Have we clearly defined the business problem we're trying to solve?"

2. Inefficient Data Handling

- "Have we thoroughly cleaned and preprocessed our data to ensure it's of high quality?"
- "Are we aware of any biases or anomalies in our data that could impact model performance?"

3. Poor Model Performance

- "Have we explored various models and performed hyperparameter tuning to ensure optimal performance?"
- "Do we have a benchmark or baseline model performance for comparison?"

4. Lack of Reproducibility

- "Can someone else easily reproduce our results based on the documentation and code we've provided?"
- "Have we documented all the steps, from data preprocessing to model selection and tuning?"

5. Failure to Detect and Mitigate Bias

- "Have we assessed our model for potential biases against certain groups or individuals?"
- "What steps have we taken to mitigate any identified biases in our model?"

6. Inadequate Evaluation

- "Are we evaluating our model on a diverse set of metrics to understand its strengths and weaknesses fully?"
- "Have we used techniques like cross-validation to ensure our model generalizes well to unseen data?"

7. Deployment Challenges

- "Is our model designed with deployment in mind, considering integration and operational needs?"
- "Have we planned for model monitoring and maintenance post-deployment?"

8. Reduced Trust and Credibility

- "Can we explain how our model makes decisions in a way that's understandable to non-technical stakeholders?"
- "Have we involved stakeholders in the evaluation process to build trust in our model's predictions?"

9. Missed Opportunities for Improvement

- "Are we continuously monitoring model performance and seeking opportunities for improvement?"
- "How responsive are we to new data or changing business needs?"

10. Legal and Ethical Implications

- "Have we considered the legal and ethical implications of our data sources and modeling techniques?"
- "Are we compliant with relevant privacy laws and ethical guidelines in our data science practices?"



My Website



My LinkedIn



My Youtube

It's important to realize that progress is Not Linear in a Data Science project.

Data Science projects are cyclical and experimental in nature.

As your understanding of the data and problem increase, you'll often revisit earlier approaches and make adjustments.

You may even modify the way you're asking the question to make your outcome more actionable.

Because of this, it's possible to jump back and forth between virtually any of the steps.

DATA SCIENCE

Steps for IMPACT



BUSINESS
UNDERSTANDING

DATA
UNDERSTANDING
& EDA

DATA
PREPARATION

MODEL
FINE-TUNNING

MODELING

FEATURE
ENGINEERING

EVALUATION
&
COMPARISON

DEPLOYMENT

EXPLAIN-
ABILITY

FAIRNESS

GOAL
REACHED



1. Business Understanding

While many teams hurry through this phase, establishing a strong business understanding is like building the foundation of a house – absolutely essential.

Steps

1.1 Determine Business Objectives

1.2 Assess Situation

1.3 Determine Data Science Goals

1.4 Produce Project Plan



2. Data Understanding & EDA

Steps

2.1 Collect Initial Data

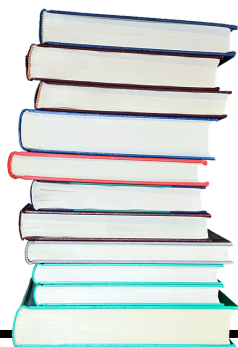
2.2 Describe Data

2.3 Explore Data

2.4 Verify Data Quality

Python Libraries

Plotly
D-Tale
ydata-profiling
Missingno
Dataprep
Lux
Holoviews
PandasGU
NumPy
Pandas
Matplotlib
Seaborn



3. Data Preparation

Steps

3.1 Select Data

3.2 Clean Data

3.3 Construct Data

3.4 Integrate Data

3.5 Format Data

Python Libraries

Feature-engine

Fancyimpute

Missingno

Category Encoders

Imbalanced-learn

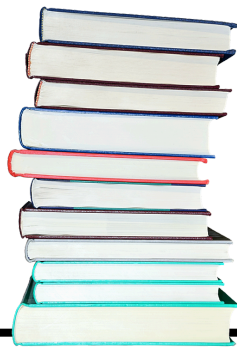
PyOD

Great Expectations

Pandas-profiling

Optimus

DataCleaner



4. Feature Engineering

Enhance model performance through the creation, selection, and transformation of features.

Steps

4.1 Feature Creation

4.2 Feature Transformation

4.3 Feature Selection

4.4 Feature Encoding

4.5 Feature Scaling

4.6 Feature Evaluation

Python Libraries

Featuretools

TSFresh (Time Series Fresh)

Polars

Category Encoders

Deep Feature Synthesis (DFS) in Featuretools

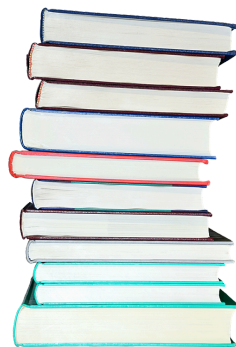
PyCaret

Boruta

sklearn-pandas

Optimus

AutoFeat



5. Modelling

Steps

5.1 Select Modeling Technique

5.2 Generate Test Design

5.3 Build Model

5.4 Assess Model

Python Libraries

Scikit-learn

TensorFlow

PyTorch

XGBoost

LightGBM

spaCy

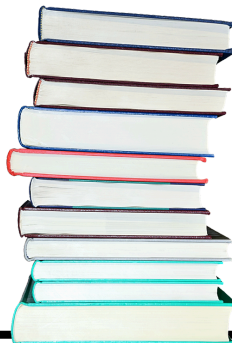
CatBoost

H2O.ai

Keras

Fast.ai

Transformers (by Hugging Face)



6. Model Fine-Tuning

Steps

6.1 Preparation

6.2 Hyperparameter Space Definition

6.3 Tuning Strategy Selection

6.4 Experimentation Setup

6.5 Execution

6.6 Evaluation and Selection

6.7 Refinement (Optional)

6.8 Finalization

6.9 Deployment Preparation

Python Libraries

Auto-Sklearn

TPOT (Tree-based Pipeline Optimization Tool)

H2O AutoML

AutoKeras

MLBox

Optuna

NNI (Neural Network Intelligence)

AutoGluon

Ray Tune

Determined AI

Hyperopt

Scikit-Optimize (skopt)

Spearmint

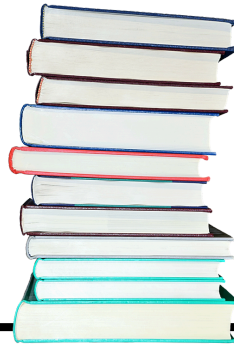
Bayesian Optimization

Nevergrad

BOHB (Bayesian Optimization and Hyperband)

GPflowOpt

Determined AI



7. Evaluation

Steps

7.1 Evaluate Results

7.2 Review Process

7.3 Determine Next Steps

Python Libraries

Scikit-learn

Statsmodels

Yellowbrick

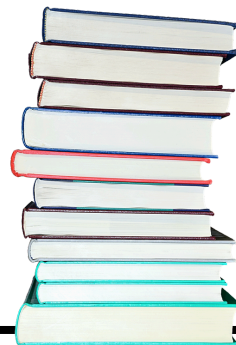
MLflow

TensorBoard

Weights & Biases

Optuna

Scikit-plot



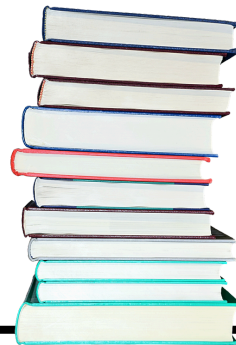
8. Deployment

Steps

- 8.1 Plan Deployment
- 8.2 Plan Monitoring and Maintenance
- 8.3 Produce Final Report
- 8.4 Review Project

Python Libraries

Streamlit
Flask
Django
FastAPI
Dash
Tornado
Bottle
Pyramid
Voilà
Anvil



9. AI Explainability

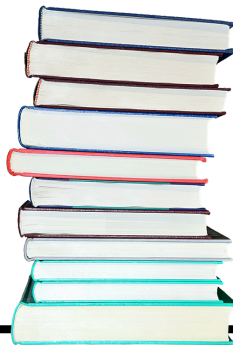
AI Explainability ensures that model decisions can be understood, questioned, and validated.

Steps

- 9.1 Understanding Explainability Needs
- 9.2 Select Explainability Techniques
- 9.3 Implement Explainability
- 9.4 Validate Explainability Outcomes
- 9.5 Monitor and Update Explainability
- 9.6 Document and Share Explainability Insights

Python Libraries

LIME (Local Interpretable Model-agnostic Explanations)
SHAP (SHapley Additive exPlanations)
Alibi
InterpretML
ELI5
What-If Tool
Skater
AIX360 (AI Explainability 360)
Fairlearn
PyTorch Captum
DALEX



10. AI Fairness

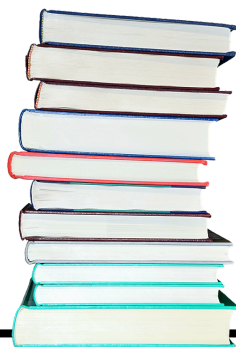
AI fairness is about developing ethical and trustworthy AI systems and preventing biases.

Steps

- 10.1 Understanding Fairness Requirements
- 10.2 Assess Data for Bias
- 10.3 Select Fairness Metrics and Techniques
- 10.4 Implement Fairness Measures
- 10.5 Validate Fairness Outcomes
- 10.6 Monitor and Update for Fairness
- 10.7 Document and Share Fairness Efforts

Python Libraries

Fairlearn
AI Fairness 360 (AIF360)
What-If Tool
EthicalML/XAI
Themis-ml
FairTest
Scikit-fairness
Fairness Comparison
Fairkit-learn
DEON



YOUR CHALLENGE

DO YOU FIND IT CHALLENGING TO APPLY THEORETICAL
AI AND DATA SCIENCE KNOWLEDGE TO REAL-WORLD SCENARIOS?

ARE YOU OVERWHELMED BY THE VAST AMOUNT OF
AI RESOURCES, UNSURE WHERE TO START OR HOW TO PROGRESS
EFFECTIVELY?

I HAVE THE RIGHT TRAINING FOR YOU.

**AI Solutions Mastery bridges the gap
between Theory and Practice by
focusing on real-world applications.**



AI Solutions Mastery

AI Practitioner Plus

You Learn

- End-to-End Data Science Implementation with Real-World Data (10-Steps)
- Advanced Feature Engineering (incl. Graph Analytics)
- Handling Imbalanced Data and Synthetic Data Generation (GANs)
- Supervised Learning (XGBoost and more)
- Deep Learning (TensorFlow Keras)
- Unsupervised Anomaly Detection (Isolation Forest, AutoEncoders)
- Unsupervised Clustering (DBSCAN, Hierarchical Clustering)
- AI Explainability and Fairness (SHAP, FairLearn)
- Model Deployment (Streamlit)

HANDS-ON TRAINING TO

Build a Real-World AI Product in 4 Weeks

From Problem to Product

20+ Years of My AI
experience in One
Training

Hands-On Training

AI Solution Mastery



VIEW TRAINING



References

<https://github.com/dslp/dslp/blob/main/steps.md>

<https://towardsdatascience.com/7-evaluation-metrics-for-clustering-algorithms-bdc537ff54d2>

<https://towardsdatascience.com/three-performance-evaluation-metrics-of-clustering-when-ground-truth-labels-are-not-available-ee08cb3ff4fb>

<https://medium.datadriveninvestor.com/evaluation-metrics-for-clustering-96dcdbea437d>

<https://towardsdatascience.com/a-comprehensive-beginners-guide-to-the-diverse-field-of-anomaly-detection-8c818d153995>

GitHub: [feature-engine/feature_engine](#)

GitHub: [iskandr/fancyimpute](#)

GitHub: [ResidentMario/missingno](#)

GitHub: [scikit-learn-contrib/category_encoders](#)

GitHub: [scikit-learn-contrib/imbalanced-learn](#)

GitHub: [yzhao062/pyod](#)

GitHub: [great-expectations/great_expectations](#)

GitHub: [pandas-profiling/pandas-profiling](#)

GitHub: [ironmussa/Optimus](#)

GitHub: [FeatureLabs/featuretools](#)

GitHub: [blue-yonder/tsfresh](#)

GitHub: [pola-rs/polars](#)

GitHub: [scikit-learn-contrib/category_encoders](#)

GitHub: [pycaret/pycaret](#)

GitHub: [scikit-learn-contrib/boruta_py](#)

GitHub: [scikit-learn-contrib/sklearn-pandas](#)

GitHub: [hi-primus/optimus](#)

GitHub: [cod3licious/autofeat](#)

GitHub: [tensorflow/tensorflow](#)

GitHub: [pytorch/pytorch](#)

GitHub: [dmlc/xgboost](#)

GitHub: [microsoft/LightGBM](#)

GitHub: [catboost/catboost](#)

GitHub: [h2oai/h2o-3](#)

GitHub: [keras-team/keras](#)

GitHub: [fastai/fastai](#)

GitHub: [optuna/optuna](#)

GitHub: [explosion/spaCy](#)

GitHub: [huggingface/transformers](#)



References

GitHub: [scikit-learn/scikit-learn](#)
GitHub: [statsmodels/statsmodels](#)
GitHub: [DistrictDataLabs/yellowbrick](#)
GitHub: [mlflow/mlflow](#)
GitHub: Part of TensorFlow: [tensorflow/tensorflow](#)
GitHub: [wandb/client](#)
GitHub: [optuna/optuna](#)
GitHub: [reiinakano/scikit-plot](#)
GitHub: [automl/auto-sklearn](#)
GitHub: [EpistasisLab/tpot](#)
GitHub: [h2oai/h2o-3](#)
GitHub: [keras-team/autokeras](#)
GitHub: [AxeldeRomblay/MLBox](#)
GitHub: [microsoft/nni](#)
GitHub: [awslabs/autogluon](#)
GitHub: [ray-project/ray](#)
GitHub: [determined-ai/determined](#)
GitHub: [hyperopt/hyperopt](#)
GitHub: [scikit-optimize/scikit-optimize](#)
GitHub: [ray-project/ray](#)
GitHub: [fmfn/BayesianOptimization](#)
GitHub: [facebookresearch/nevergrad](#)
[automl/HpBandSter](#)
GitHub: [GPflow/GPflowOpt.](#)
GitHub: [determined-ai/determined](#)
GitHub: [HIPS/Spearmint.](#)
GitHub: [streamlit/streamlit](#)
GitHub: [pallets/flask](#)
GitHub: [django/django](#)
GitHub: [tiangolo/fastapi](#)
GitHub: [plotly/dash](#)
GitHub: [tornadoweb/tornado](#)
GitHub: [bottlepy/bottle](#)
GitHub: [Pylons/pyramid](#)
GitHub: [voila-dashboards/voila](#)
Web: [Anvil](#)



References

GitHub: [oracle/Skater](#)
GitHub: [Trusted-AI/AIX360](#)
GitHub: [fairlearn/fairlearn](#)
GitHub: [pytorch/captum](#)
GitHub: [ModelOriented/DALEX](#)
GitHub: [marcotcr/lime](#)
GitHub: [slundberg/shap](#)
GitHub: [SeldonIO/alibi](#)
GitHub: [interpretml/interpret](#)
GitHub: [TeamHG-Memex/eli5](#)
GitHub: [fairlearn/fairlearn](#)
GitHub: [Trusted-AI/AIF360](#)
GitHub: [EthicalML/xai](#)
GitHub: [ayrna/themis-ml](#)
GitHub: [columbia/fairtest](#)
GitHub: [scikit-learn-contrib](#)
GitHub: [algofairness/fairness-comparison](#)
GitHub: [fairkit-learn/fairkit-learn](#)
GitHub: [drivendataorg/deon](#)